

DOCKER

Modül 17.6.5

Production İçin Compose Best Practices

Canlı ortam için güvenli ve sürdürülebilir pratikler

Data Science RoadMap

Alican Kaya

github.com/AlicanKaya192

Geliştirme vs Canlı Ortam

Geliştirme ortamında bind mount'lar ve live-reloading kullanarak hızlı iterasyon yaparız. Canlı ortamda (Production) ise güvenlik, kararlılık ve performans önemlidir.

1. Multiple Compose Dosyaları Kullanımı

Base (temel) bir `docker-compose.yml` oluşturulur. Ardından ortama özel (dev, prod) dosyalarla bu temel dosya üzerine eklentiler/ezmeler yapılır (Compose Overrides).

```
yaml

# 1. Temel dosya (docker-compose.yml)
services:
  web:
    image: my_app

# 2. Geliştirme dosyası (docker-compose.dev.yml)
services:
  web:
    volumes:
      - ./app # Kod senkronizasyonu

# 3. Üretim dosyası (docker-compose.prod.yml)
services:
  web:
    restart: always # Çökünce yeniden başlat
```

Başlatmak için (Prod): `docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d`

2. Restart Politikaları Ekleyin

Konteynerler bilinmeyen hatalar yüzünden çökebilir veya sunucu yeniden başlayabilir. Her servis için bir restart politikası belirleyin.

```
yaml

services:
  web:
    image: nginx
    restart: unless-stopped
```

3. Kaynak Limitleri Koyun (cgroups)

Bir servis memory leak (RAM sızıntısı) yaparsa tüm sunucuyu dondurabilir. Bu yüzden CPU ve Memory limitleri koyun.

```
yaml

services:
  db:
    image: postgres
    deploy:
      resources:
        limits:
          cpus: '1.5'
          memory: 1G
```

4. Network İzolasyonu (DMZ)

Veritabanınıza port yönlendirmesi (-p 5432:5432) YAPMAYIN. Sadece backend servisi aynı network üzerinden doğrudan (örn: port 5432 kullanarak) veritabanına ulaşmalıdır. Böylece DB dış dünyaya kapalı olur.